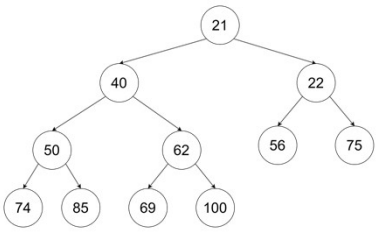


Priority Queue

1

Priority Queue

Una fila donde la eliminación depende de la prioridad y no del orden de inserción.

Java lo implementa de forma eficiente usando un heap binario.

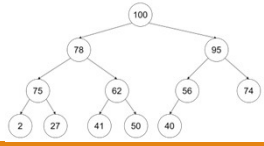
2

Heap Repaso

3

Heaps

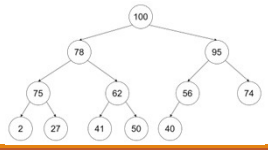
- Los heaps son árboles completos que siguen la propiedad del heap.
- Se discutió el heap máximo, donde un nodo principal debe ser mayor o igual que sus hijos.
- También se mencionó el heap mínimo, donde el nodo principal debe ser menor o igual a sus hijos.



4

Heaps

- Un Heap, Al igual que un `Stack` o un Queue, se centra principalmente en un valor de la estructura.
- En este caso, nos enfocamos en la raíz, que es el valor máximo (o mínimo).
- Solo podemos eliminar u obtener la entrada en la raíz del Heap.

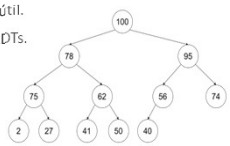


5

5

Heaps

- Se discutió cómo esta es una estructura eficiente, ya que la mayoría de las operaciones son O(log n).
- Pero al igual que `Stacks` y `Queues`, por muy eficientes que sean, su uso funciona para situaciones muy específicas.
- Un Heap, como un ADT independiente no es muy útil.
- Sobre todo lo utilizaremos como base para otros ADTs.
  - Uno de estos casos es el `Priority Queue`.



6

6

Priority Queue

7

7

Priority Queues

- **Queues** son estructuras que tienen una política de almacenamiento FIFO.
- Se discutió que este ADT es útil para los casos en los que se desea simular una fila de espera.
- Se podría decir que el queue prioriza la hora de llegada.
- Pero, ¿es siempre la mejor manera de establecer una fila de espera?

8

8

Priority Queues

Considere este ejemplo:

- Habíamos mencionado anteriormente que una **queue** es lo que normalmente usamos para configurar una fila de espera de impresora.
- Y que tenía más sentido que usar un stack, ya que esto correría el riesgo de que los documentos más antiguos nunca se imprimieran.
- Pero, ¿qué pasaría si en el queue tuviéramos varios documentos con 50+ páginas y al final del queue un documento con solo 2 páginas en espera.

9

9

Priority Queues

Ejemplo:

- Al ser un **Queue**, Garantizamos que todos estos documentos llegarán a ser impresos (mejor que el Stack).
- Pero puede ser frustrante tener que esperar tanto tiempo para imprimir un documento tan corto.
- ¿Hay alguna manera de organizar mejor el queue de espera?

10

10

Priority Queues

- En situaciones como estas, podríamos usar un **Priority Queue**.
- En lugar de almacenar datos en una política FIFO, organiza los datos en función de un valor de prioridad.
  - Esta prioridad dictará cuál es el próximo elemento al que tenemos acceso.
  - Cuanto mayor sea la prioridad, más cerca del frente estará el elemento.

11

11

Priority Queues

- Teniendo en cuenta esto, si añadimos un nuevo elemento y tiene una prioridad más alta que todo lo demás, se añade al principio.
- Cuando quitemos el elemento del frente, el siguiente elemento será el que tenga la siguiente prioridad más alta.
- Esto es como una lista ordenada, donde los datos se ordenan de forma ascendente (o descendente), en este caso, de prioridad.
- La diferencia es que en una lista tenemos acceso a todos los valores, mientras que en el Queue solo tenemos acceso a la prioridad más alta (por ejemplo, la posición 0).

12

12

Priority Queues

Pero, ¿qué dicta esta prioridad?

- Como siempre, depende de los datos.
- Para el ejemplo del queue de la impresora, podría ser:
  - Hora de llegada(FIFO)
  - Número de páginas
  - Tamaño del archivo

13

13

Priority Queues

Pero, ¿qué dicta esta prioridad?

- En general, para el Priority Queue Por lo general, usamos Pares key-value para almacenar datos, no solo valores como en los queues normales.
- La intuición es que la clave representa el valor de prioridad, y el valor son los datos en sí.
- Al igual que el heap, podemos repetir keys en esta estructura.
- Esto significa que 2 elementos pueden tener el mismo nivel de prioridad.

14

14

Priority Queues

¿Cómo ordenamos las prioridades??

- Podemos utilizar datos comparables o utilizar comparadores.

¿Organizamos esos datos en orden ascendente o descendente??

- Depende de lo que establezcamos como prioridad (key).
- Normalmente, se utiliza el orden ascendente.
- Lo que significa que cuanto más pequeña sea el key, mayor será la prioridad.

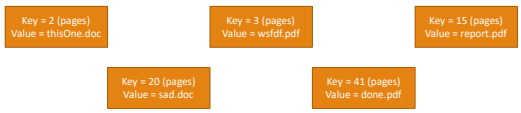
15

15

Priority Queues

Ejemplo

- Supongamos que tenemos estos documentos para una impresora y establecemos el número de página como prioridad.



¿En qué orden se almacenarán en elPriority Queue?

16

16

Priority Queues

Ejemplo

- Supongamos que tenemos estos documentos para una impresora y establecemos el número de páginas como prioridad.
- Esto daría como resultado:

Key = 2 (pages) Value = thisOne.doc	Key = 3 (pages) Value = wsfdf.pdf	Key = 15 (pages) Value = report.pdf	Key = 20 (pages) Value = sad.doc	Key = 41 (pages) Value = done.pdf

17

17

Priority Queue ADT

18

18

Métodos Java

Método	Descripción
`add(e)`	Inserta elemento
`offer(e)`	Inserta elemento (preferido en queues)
`peek()`	Devuelve el elemento superior (sin eliminación)
`poll()`	Elimina y devuelve el elemento superior
`remove()`	Elimina elemento superior
`isEmpty()`	Comprueba si está vacío

19

19

Ejemplo (default: min-heap)

```
import java.util.PriorityQueue;
public class Main {
  public static void main(String[] args) {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    pq.add(30);
    pq.add(10);
    pq.add(20);
    System.out.println(pq); //no necesariamente está ordenado [10, 30, 20]
    while (!pq.isEmpty()) {
      System.out.println(pq.poll());
    } } }
```

10, 20, 30

20

20

Max-heap (personalizado)

Por defecto es un min-heap, pero puedes invertirlo:

```
PriorityQueue<Integer> pq = new PriorityQueue<>(Collections.reverseOrder());
```

21

21

Ejemplo con objetos

```
import java.util.*;

class Student {
   String name;
   int grade;

   Student(String n, int g) {
      name = n;
      grade = g;
   }
}
```

```
public class Main {
   public static void main(String[] args) {

      PriorityQueue<Student> pq =
         new PriorityQueue<>(
            (a, b) -> b.grade - a.grade
         );

      pq.add(new Student("Ana", 90));
      pq.add(new Student("Luis", 70));
      pq.add(new Student("Maria", 95));

      while (!pq.isEmpty()) {
         Student s = pq.poll();
         System.out.println(s.name + " " + s.grade);
      }   }  }
```

22

22

```
Comparator<Student> comp = new Comparator<Student>() {
   public int compare(Student a, Student b) {
      return b.grade - a.grade;
   }
};
```

23

23

Ejemplo con objetos - el comparador local dentro del main

```
import java.util.*;
class Student {
   String name;
   int grade;
   Student(String n, int g) {
      name = n;
      grade = g;
   } }
public class priori {
   public static void main(String[] args) {
      Comparator<Student> comp = new Comparator<Student>() {
         public int compare(Student a, Student b) {
            return b.grade - a.grade;
         }      };
```

```
PriorityQueue<Student>  pq =
      new PriorityQueue<>(comp);
   pq.add(new Student("Ana", 90));
   pq.add(new Student("Luis", 70));
   pq.add(new Student("Maria", 95));
   while (!pq.isEmpty()) {
      Student s = pq.poll();
      System.out.println(s.name + " " + s.grade);
   }   }  }
```

24

24

Priority Queue Implementación

- Es posible que haya notado que el comportamiento es casi idéntico al de un min Heap.
- Es por eso que podríamos usar un objeto Heap para implementar un `Priority Queue`.

25

25

Priority Queue Implementación

- Pero, ¿cuál es la diferencia entre un Priority Queue y un Heap?
- Un Heap es una implementación de árbol particular. Sigue la propiedad del heap.
- Mientras que Priority Queue es una clase en java, implementada como heap.

26

26

Otras implementaciones

27

27

Implementaciones

- Algunos usos de los Priority Queue son:
  - Itinerario en un sistema operativo.
  - Ayuda en ciertos algoritmos, como el algoritmo de ruta más corta de Dijkstra (grafos), el algoritmo de prim (grafos) y algoritmo heap sort.
  - Compresión de datos (Huffman Codes)..

28

28

Implementaciones

CPU Scheduling:

- En cursos posteriores verás que muchos de los diferentes ADTs discutidos en este curso son muy utilizados para la implementación de Sistemas Operativos.
- En el caso de un`Priority Queue`, se utiliza para la programación de CPU.
- Cada vez que una computadora ejecuta una función (Un conjunto de instrucciones) A esto se le llama proceso.
- Normalmente, en una computadora tenemos muchos procesos intentando ejecutarse al mismo tiempo.

29

29

Implementaciones

Programación de CPU:

- El orden en el que ocurren estos procesos es importante, ya que hay ciertas operaciones que queremos que se realicen de inmediato.
  - actualizar la pantalla cuando movemos el mouse.
  - Procesando I/O.
  - Control de una excepción o error.
- Pero no todos los procesos son iguales, algunos toman más tiempo que otros.

30

30

Implementaciones

Programación de CPU:

- El objetivo de la programación del CPU es programar el orden en el que la CPU ejecuta cada proceso.
- Una forma en que podríamos programar los procesos es por orden de llegada.
- Pero esto nos llevaría al mismo problema que antes, los procesos rápidos podrían retrasarse o, lo que es peor, las tareas importantes podrían estancarse.
- Para solucionar esto, podemos asignar prioridades a cada proceso y, en función de eso, el CPU determina qué ejecutar a continuación.

31

31